Smart Transportation Platform for Private Transportation

Vially Kazadi Mutombo
Soongsil University
Seoul, Republic of Korea
mutombo.kazadi@gmail.com

Yena Lee
Soongsil University
Seoul, Republic of Korea
ynlee.oslab@gmail.com

Heejin Kim
Soongsil University
Seoul, Republic of Korea
khj.oslab@gmail.com

Younggwan Kim
Soongsil University
Seoul, Republic of Korea
ygkim.oslab@gmail.com

Natalia Weronika Debska
Soongsil University
Seoul, Republic of Korea
natadebska@hotmail.com

Jiman Hong*
Soongsil University
Seoul, Republic of Korea
jiman@ssu.ac.kr

ABSTRACT

The diversity of technologies in today's world has made Smart city a reality and contribute to improving living conditions. Smart transportation is one of the smart city aspects that aims to solve many problems in cities such as traffic congestion problems, lack of parking lot and others. In this paper, we propose a smart transportation platform for analyzing and designing smart private transportation applications. The proposed model can be served as a blueprint for smart parking application projects in order to solve the lack of parking slot problems. The proposed model can be used for deriving an optimal routing model based on the traffic congestion in order to alleviate traffic congestion during peak time and reduce the travel time, fuel consumption and CO_2 emission. The evaluation results show that the proposed model can improve the quality indexes and with the derived optimal routing model, it can have improvements in terms of travel time, fuel consumption and CO_2 emission that have been decreased.

CCS CONCEPTS

• **Computer systems organization → Internet of Things, Software Platforms**• → Software Model

KEYWORDS

Smart transportation, Software design, Optimal routing framework, Smart parking Framework

ACM Reference format:

Kazadi Mutombo Vially, Yena Lee, Heejin Kim, Younggwan Kim, Natalia Weronika Debska, Jiman Hong. 2020. Smart Transportation Platform for Private Transportation. In *The 35th ACM/SIGAPP Symposium on Applied Computing (SAC '20), March 30-April 3, 2020, Brno, Czech Republic*, 8 pages. https://doi.org/10.1145/3341105.3374043

* Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SAC '20, March 30-April 3, 2020, Brno, Czech Republic
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-6866-7/20/03…$15.00
https://doi.org/10.1145/3341105.3374043

1 INTRODUCTION

Smart transportation is one of the key features of a smart city that aim to solve the transportation problems in the cities and provide better transportation management services such as traffic management, smart taxi system, parking management, passengers' management. Some developed countries such as the United States of America (USA) lost $305 billion economically and time wasted estimated at 34hours each year as of 2017 due to traffic problems [12].

Traffic congestion is one of the main transportation problems in the city as it makes transportation inconvenient. Some cities invested a lot of money in order to solve the traffic congestion however, with the growth of the population, the traffic congestion in cities is still increasing day by day. Various solutions have been proposed to reduce traffic congestion in smart transportation such as the construction of high ways, traffic light synchronization system or a navigation system.

Smart transportation to solve traffic congestion is associated with both private and public transportation, yet the private transportation's traffic keeps increasing every day. In fact, the share of public vehicles in large Indian cities actually declined from some 70 percent in 1994 to almost 40 percent in 2007[20].

K. Chopra et al. presented that most vehicles in cities are private, the population growth keeps increasing the number of private vehicles in cities. Hence, this has become the main reason for traffic congestion problems, in [10]. Therefore, solving the various private transportation issues can reduce significantly many other problems that affect city traffic, such as traffic congestion, parking management, CO_2 emissions, fuel consumption, and time-wasting.

Private transportation issues including lack of parking slots and lack of optimal routing solutions are some of the most discussed topics in smart transportation. Many people have difficulty in finding a free parking slot, which results in additional delays and extra traffic congestion. Similarly, in the central areas

of large cities, drivers can spend 20 minutes looking for parking spaces affecting 10% of the traffic [2, 3].

In most cases, the traffic congestion issues are caused by private vehicles rather than public due to the lack of optimal routing solution. It is caused by public transportation, where routes are predefined, as the driver must pick up passengers at each station. Whereas, in private transportation, the driver is free to decide the route from the trip's origin to its destination. There, smart traffic applications need to provide recommendations for optimal routes based on traffic congestion to private transportation drivers. In addition, developing smart parking applications for smart cities requires a good software design as the basis to be able to handle huge demand during peak time.

In this paper, we propose an efficient smart transportation platform for designing private vehicle transportation applications in smart cities. The proposed platform focuses on the private transportation system, aiming to solve the problems faced by most private vehicle drivers, such as lack of parking slots and lack of routing models during peak hours.

The subsequent sections of this paper are organized as follows. Section 2 presents related works. Section 3 presents the proposed private transportation platform to analyze and design private vehicle transportation applications and Section 4 presents the evaluation results of the proposed platform. Finally, Conclusion remarks are given in Section 5.

2 RELATED WORK

Considerable research efforts [1,2,4,5,8,12] have been made to solve transportation problems to improve and built the smart transportation system for a smart city. M. Pečar and G. Papa in [2] presented various problems related to transportation and mobility in cities and their potential solutions in a smart city. M. Pečar and G. Papa identified five major problems such as traffic congestion, parking difficulties, longer commuting, public Transportation inadequacy, and freight distribution.

In [4], M. Lubis et al. revealed that a daily estimation of 30% of vehicles on the road in the downtown of big cities is wasting more than 7.8 minutes looking for a parking slot. It may cause a lot of problem such as fuel consumption, CO_2 emission, time waste as well as other drivers time waste due to the traffic congestion occurs.

In [8], M. Balta and I. Özçelic proposed a traffic signaling optimization using the Ant Colony algorithm in order to alleviate traffic congestion in an urban isolated intersection structure. The solution reduced the delay time of vehicles injunction, fuel consumption, noise as well as CO_2 emission. Balta and Özçelic aimed to solve the traffic congestion problem, which we are also solving in this paper. In contrast, the authors focused on traffic signalization, which is also another aspect of the problem, while in this paper we are focusing on reducing the congestion caused by private vehicles.

Yadavalli in [1] proposed a smart parking system based on the Internet of Things (IoT), including sensor devices that enabled the real-time update of the parking slot's occupancy. The proposed Smart Parking Web application allowed drivers to check the availability of parking slots. The implementation results showed that the application could update the parking availability. However, this solution does not include the reservation feature to allow the drivers to reserve the slot before reaching the parking area. In addition, the software design showed a low quality in terms of effectiveness, extensibility, and functionality.

A. Khanna and R. Anand in [5] proposed an IoT-based Smart Parking System. The authors focused more on the physical aspect, where they proved how an IoT-based architecture would be a good solution to provide a smart parking system allowing users to remotely check parking slot's occupancy and make a reservation. In addition, Khanna and Anand revealed the need for IoT-Cloud integration in order to deal with IoT limitations and benefits from cloud computing advantages. The implementation results showed that by using a mobile application users can check parking occupancy and make a reservation in case if a parking slot is available. Unlike our work, we propose a platform for designing smart parking applications with the ability to interconnect multiple parking spots within a smart city and integration of optimal routing solutions. The proposed solution will serve as a blueprint to build smart transportation applications for private vehicles.

3 SMART TRANSPORTATION PLATFORM

A smart transportation application for private vehicles must include the optimal routing and smart parking features. When a vehicle driver wants to move from a place to another, the application should recommend the optimal route as well as the nearest parking slot to the destination to reserve a parking slot. Therefore, our proposed private transportation platform consists of two parts, such as the smart parking framework and the congestion-based optimal routing framework.

3.1 Smart Parking Framework

The proposed smart parking framework is done using the Clean Architecture methodology [6]. Starting with the specification of Enterprise business rules, the smart parking application stakeholder is the project manager and four entities: the driver, the vehicle, the parking area, and the parking slot. In this module, the idea is that, when the driver wants to move from one place to another, the application will recommend the nearest available parking lot. That will allow the driver to directly reserve the parking slot and prevent the driver from spending time looking for it as he reaches the destination.

The next phase is the definition of the application business rules or use case layer. The software engineer as the stakeholder of this activity must identify the software application requirements and present the use case diagram to show the data flow between entities in the system. Therefore, at this phase, we present the use case of the proposed smart parking system in Fig. 1 and Fig. 2.

Fig. 3 shows the class diagram for the reservation feature of Smart Parking. From this diagram, a driver who intends to travel from a source point *A* to a destination point *B* gets the recommendation of the nearest parking lots with an available slot. We suggest the nearest parking lots to be found using the K-

Nearest Neighbour (KNN) algorithm [21] as shown in Fig. 4. However, the smart parking slot availability will be updated automatically from sensor nodes. The driver can then make a reservation and pay online or at arrival. The system also offers the driver an option to save in his wishlist parking lots, which were frequently visited for future reservations.

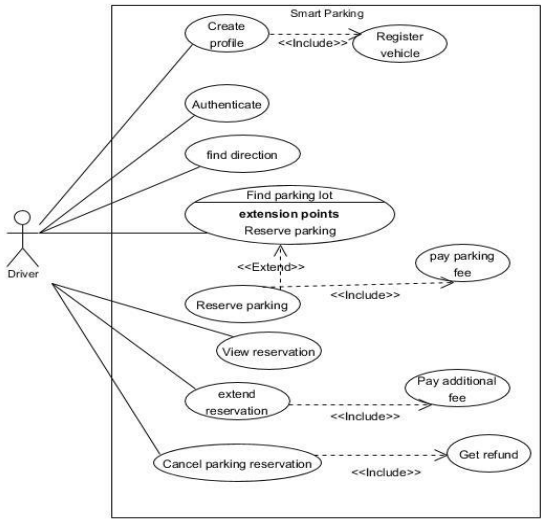

Figure 1: Use Case Diagram of Smart Parking

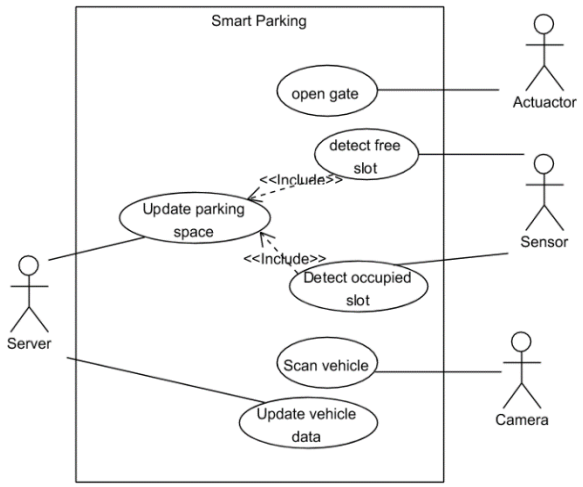

Figure 2: Use Case Diagram of Parking slot's occupancy update

Fig. 4 shows the class diagram of the smart parking system based on IoT devices. The parking lot is equipped with sensors, camera, and actuators at the entrance and exit gates. Each parking slot is equipped with a position sensor that allows updating the slot occupancy status.

The next phase is the definition of Interface Adapters. This phase consists of defining the intermediary layer of the smart parking application that controls the flow of data between the application and frameworks. This layer is similar to the controller in the MVC (Model View Controller). The stakeholder of this activity is the Software engineer and the application developer who are responsible for the activities such as presentation of data, delivery mechanisms, view of data and data management. Moreover, in order to deal with the tsunami of data generated by IoT networks, Khanna and Anand proposed in [5] the IoT-Cloud integration. For this purpose, data analytics can be listed as an activity of this phase.

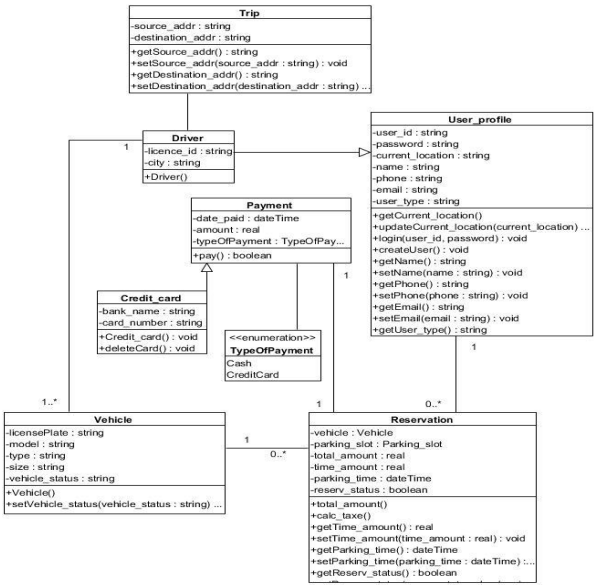

Figure 3: Class Diagram of Parking Reservation

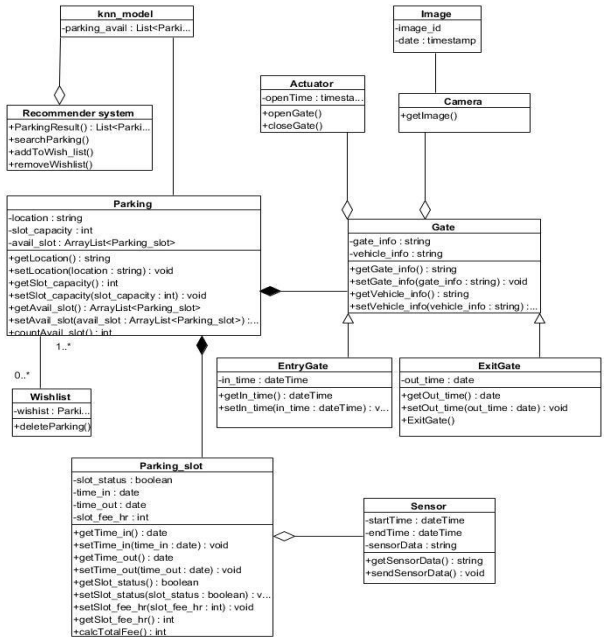

Figure 4: **Class Diagram of Smart Parking**

The final phase is the specification of the frameworks and vehicle drivers. This phase has three stakeholders such as the data engineers, the IoT experts, and the hardware engineers. In the case of private transportation, data engineers should identify the relevant solutions for data collection, storage, and processing. The IoT experts along with the hardware engineers should also identify the required devices such as sensors, actuators, network infrastructures and devices, cameras and any other devices to be utilized by the smart transportation applications. At this stage, the software engineers should identify all the external systems that interact with the application such as GPS, web-mapping service, for example, Google Maps and traffic synchronization systems.

3.2 Congestion-based Optimal Routing Framework

An optimal routing framework should be applied to private transportation, as well as the Taxi transportation system, in order to alleviate the traffic congestion in the city. Therefore, an optimal routing framework should be based on the real-time traffic congestion state.

We used the Greedy approach to compute the optimal routing framework with consideration of factors such as distance between the source and destination, congestion factor and the real-time average speed. The congestion factor is a coefficient of traffic congestion status that has an impact on the vehicle speed. The more congested is the road the slower the speed. Thus, the vehicle travel time and the CO_2 emission increase. The congestion factor (CF) of each path can be computed using Eq. (1)

$$\mathrm{CF} = \mathrm{CS} * \mathrm{R}_k \qquad \text{with k=1,2,3,…,n} \tag{1}$$

where CS is the congestion status of each road, *p* is the total number of roads that compose a path and R_k is the ratio of the road k of a given path.

Then the estimated time (ET) to travel within the road k can be computed by using Eq. (2).

$$\mathrm{ET_k} = \frac{\mathrm{D}}{\mathrm{S} * \mathrm{CF}} \tag{2}$$

where *D* is the distance the road and *S* is the maximal speed of the road, as each road has its maximal speed requirement. As shown in the Eq. (2), the congestion factor (CF) is inversely proportional to the travel time. The total travel time to reach the destination using a given route *r* can be determined by the summation of the estimated time of n roads composing the routing is given by:

$$\mathrm{TT_r} = \sum_{\mathrm{i=1}}^{\mathrm{n}} \mathrm{ET_i} \tag{3}$$

Finally. the optimal path by the minimal travel time of all the possible paths from the departure point to the destination is given by:

$$\mathrm{TT_{opt}} = \min\{\mathrm{TT_1}\ , \mathrm{TT_2}, \mathrm{TT_3}, \dots, \mathrm{TT_n}\} \qquad \text{with } \mathrm{n} > 0 \tag{4}$$

In practice, in order to compute the optimal routing path, which has minimal travel time, we use the Dynamic Programming algorithm [15]. Therefore, the vertex weight corresponds to the estimated time to travel within the vertex and can be expressed by using Eq. (2) in the proposed private transportation platform. The cost matrix to get the minimal routing to reach the destination is given by

$$\mathrm{TT} = \begin{bmatrix} \mathrm{ET_{1,1}} & \mathrm{ET_{1,2}} & & \mathrm{ET_{1,j}} \\ \mathrm{ET_{2,1}} & \mathrm{ET_{2,2}} & \cdots & \mathrm{ET_{2,j}} \\ \mathrm{ET_{3,1}} & \mathrm{ET_{3,2}} & & \mathrm{ET_{3,j}} \\ & \vdots & \ddots & \vdots \\ \mathrm{ET_{i,1}} & \mathrm{ET_{i,2}} & \cdots & \mathrm{ET_{i,j}} \end{bmatrix} \tag{5}$$

where, $i, j > 0$ and $i=j$, the number of nodes in the graph, and $ET_{i,j}$ refers to the weight of the vertex between the nodes *i* and *j*. Then, the optimal routing is given by:

$$\mathrm{TT^k[i,j]} = \min\{\mathrm{TT^{k\text{-}1}[i,j]}, \mathrm{TT^{k\text{-}1}[i,k]} + \mathrm{TT^{k\text{-}1}[k,j]}\} \tag{6}$$

where *k* is an intermediate vertex.

As mentioned earlier, the optimal routing is based on the congestion factor. During peak time, some roads present a very high congestion rate than others. However, by using the shortest or fastest routing algorithm, the recommended route is likely to be the same for most drivers traveling from the same source to the same destination. Therefore, as during peak time many drivers can move from the same source and destination, the likelihood for most of them to get the recommendation of the same route is very high. Eventually, this triggers traffic congestion on some roads while others remain free.

The proposed congestion based optimal routing framework provides a load balancing of vehicle traffic to avoid traffic congestion. The proposed solution is able to detect traffic congestion on road through information collected periodically from vehicles and road detector devices. The following steps are defined to describe the proposed solution:

Step 1: Initiate the trip from source to destination

Step 2: Find possible routes

Step 3: Compute congestion factor and find the optimal routing

Step 4: Assign the optimal route to the vehicle

Step 5: Check the vehicle's current road state

Step 6: Update the vehicle's route if any congestion occurred

To find the nearest parking slot from the destination, KNN (K Nearest Neighbor) model which is based on the Euclidean Distance Algorithm can be used in order to compute the *K* nearest parking lot to be recommended to the driver with the value of *K=3* or higher.

4 EVALUATION & RESULTS

We evaluated the design quality of the proposed smart parking framework by using QMOOD (Quality Metrics for Oriented Object Design) [11] and the performance of the proposed optimal

routing framework by using SUMO (Simulator for Urban Mobility) and TraCI4Matlab for private transportation in smart cities[13].

4.1 EVALUATION OF SMART PARKING FRAMWORK

The production view of software design quality is concerned with the software component quality, which influences the software framework quality and affects the software end-product quality extensively [19]. To evaluate the quality design of the proposed framework, we have considered following software design quality metrics for the quantitative evaluation of the proposed framework by computing the design attributes shown in table 1.

Table 1: Design Properties for calculating quality metrics

Design Attributes	Description
Encapsulation: Data Access Metric (DAM)	The ratio of the number of private (protected) attributes to the total number of attributes declared in the class.
Design Size in Classes(DSC)	The number of the total number of classes in the design.
Hierarchies	It is a count of number of non-inherited classes that have children in a design.
Coupling: Direct Class Coupling(DCC)	The number of the different number of classes that are directly related by attribute declarations and message passing (parameters) in methods.
Cohesion: Cohesion Among Methods of classes(CAM)	The summation of the intersection of parameters of a method with the maximum independent set of all the parameter types in the class.
Inheritance: Measure of Functional Abstraction (MFA)	The ratio of the number of methods inherited by the total number of methods accessible by a member of the methods of the class.
Polymorphism: Number of Polymorphic Methods(NOP)	The number the methods that can exhibit polymorphism behavior.
Abstraction: Average Number of Ancestors(ANA)	Computed by determining the number of classes along all paths from the root classes to all the classes in an inheritance structure
Composition: Measure of Aggregation (MOA)	Computed by determining the number of attributes declarations whose types are user-defined classes.
Message: Class Interface Size(CIS)	The number of all public methods in a class.
Complexity: Number of Methods(NOM)	The number of all the methods defined in a class.

Following software design quality metrics which are responsible for internal quality are considered to evaluate the quality of the proposed framework.

- *Reusability*: the ability to be reapplied to a new problem effortlessly.
- *Flexibility*: the ability to adapt to any change.
- *Functionality*: the ability of a system or application to do what it is expected.
- *Extendibility*: the ability to incorporate new functionalities to the application.
- *Effectiveness:* the ability to achieve the desired functionality.

The software design quality metrics can be obtained by calculating the quality attributes by using a well-known design methodology, QMOOD as shown in Table 2. The design quality attributes are computed based on all the design properties described in Table 1.

Table 2: Quality Metrics(attibutes) formulas

Quality Metrics	Formulas
Reusability	*-0.25 * Coupling + 0.25 * Cohesion + 0.5 * Messaging + 0.5 * Design Size*
Flexibility	*0.25 * Encapsulation - 0.25 * Coupling +0.5 * Composition + 0.5 *Polymorphism*
Extendibility	*0.25 * Encapsulation - 0.25 * Coupling+0.5 * Inheritance + 0.5 * Polymorphism*
Understandability	*-0.33 * Abstraction + 0.33 * Encapsulation – 0.33 * Coupling+ 0.33 * Cohesion – 0.33 * Polymorphism – 0.33 * Complexity – 0.33 * Design Size*
Functionality	*0.12 * Cohesion + 0.22 * Polymorphism + 0.22 * Messaging +0.22 * Design Size + 0.22 Hierarchies*
Effectiveness	*0.2 * Abstraction + 0.2 * Encapsulation + 0.2 * Composition+ 0.2 * Inheritance –0.2 * Polymorphism*

As the transportation domain is very huge and includes the diversity of applications, interoperability between those applications is very important in smart cities. Therefore, we used the Clean Architecture mythology to allow an interoperable analysis of the huge and diverse smart parking applications. The calculation of quality metrics can give values of different ranges. Therefore, a normalization process will be needed in the calculation. The following steps should be performed compute the quality metrics and the overall quality index:

- Step 1: Compute all the design metrics as described in Table 1 with the class diagram (converted into a programming language) as input;
- Step 2: Normalize all the values of the design metrics;

- Step 3: Compute all the quality metrics from the design metrics using the formula presented in Table 2;
- Step 4: Compute the total quality index known as TQI by doing the summation of all quality metrics.

The computation of Step 1 can be done by using well-known tools such as Eclipse or SDMetrics to retrieve the values of design metrics. In our evaluation, we have used SDMetrics, which is a software design metrics' tool for analyzing the structure of UML models and measure quality attributes of object-oriented models [17].

Table 3 presents the values of the design metrics obtained from two designs compared to this work. The second column shows the result of the smart parking system(SPS) proposed in [1] and the third column, our proposed framework. As show in table 3, the design size of the proposed framework is 2.375 times bigger than the SPS, as we have more classes as shown in Fig. 3 and Fig. 4. The design metrics like NOH, ANA, MFA, NOP have 0 in the SPS, due to the lack of hierarchical relationship and aggregation between entities in the SPS. In addition, the design metrics such as messaging and complexity are proportional to the design size. Therefore SPS had a value of 28 for both messaging and complexity, while the proposed framework had the value of 70. Note that Messaging and Complexity are the same if all methods defined in classes are public.

Table 3: Comparison of the Design Metrics

DESIGN METRICS	Smart Parking System [1]	Proposed Framework
	Value	Value
Design Size(DSC)	8	19
Hierarchies(NOH)	0	3
Abstraction(ANA)	0	4
Encapsulation(DAM)	1	1
coupling(DCC)	0	0
Cohesion(CAM)	1	1
Composition(MO)	0	6
Inheritance(MFA)	0	0.33
Polymorphism(NOP)	0	13
Messaging(CIS)	28	70
Complexity(NOM)	28	70

The results of comparing the quality metrics as shown in Fig. 5. The quality metrics were computed by normalizing the value of each metric. The results show that the understandability metric has decreased by 22.36% because of the previous smart parking system less complex than the proposed framework. However, most of the remaining design quality metrics such as Flexibility, Extendibility, and Effectiveness which are the important decision factor for making the good design have been significantly improved in the proposed framework. Flexibility and Effectiveness metrics of the proposed framework are improved by 93.91% and 90.24% respectively.

In addition, the previous smart parking system has shown to be not extendable with the value of extendibility metric equal to 0. Whereas, it has been improved from 0 to 0.46 in the proposed framework. It means that the proposed framework is applicable to even the system in which the new features easily added to the original system.

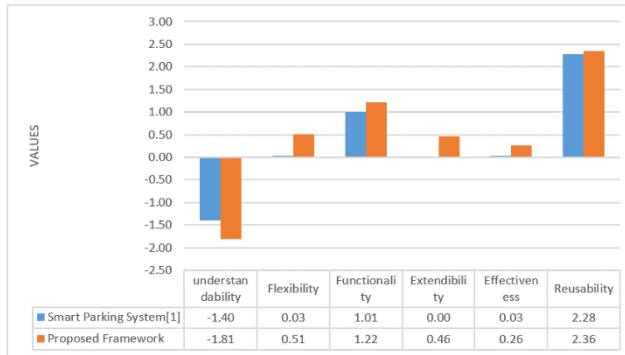

Figure 5: Comparison of Quality Metrics

Moreover, Functionality and Reusability are also improved by 17.76% 3.24% respectively. Finally, we also calculated the Total Quality Index (TQI) as described in the process step 4 as shown in Table 4.

The higher value of quality attributes of software design ensures better software quality. Bansiya and Davis in [11] defined the Total Quality Index of software design as the summation of a summation of quality metrics(attributes) of the design. This implies that a higher Total Quality Index of software design ensures a better quality of the software design.

Table 4: Total Quality Index

DESIGN	TQI
SPS	1.94
R-SPS	3.00

4.2 EVALUATION OF OPTIMAL ROUTING FRAMEWORK

In order to evaluate the performance of the proposed optimal routing framework, we conducted experiments using SUMO, which is an open-source, highly portable, microscopic, and continuous road traffic simulator [14]. We also TraCI4Matlab which is an Application Programming Interface (API) developed in Matlab to allow users to interact with a SUMO running Simulation through TRACI Library using a TCP/IP connection[18]. Fig. 6 is the sample road map used in our simulation.

To evaluate the traffic congestion, we used the Multi-Entry-Exit detectors along the roads to quantify the traffic. Besides, we collected periodically information from vehicles in real-time simulation such as vehicle's speed, waiting time, fuel consumption and CO_2 emission.

We evaluated and compared the performance of our optimal routing framework against Dijkstra's algorithm using different evaluation measures and different test cases. Our framework was tested in Matlab and interacting with the simulation in real-time using Traci4Matlab as mentioned earlier in the literature. And we used Dijkstra's algorithm which is available in SUMO. We performed the test through many test cases in which, we were increasing the number of vehicles in the simulation. We got traffic congestion at some points and at the same time step, we launched two vehicles with the same source and destination. One vehicle using Dijkstra's algorithm and the other one our routing approach.

Three types of data were collected from the simulation including the travel time, fuel consumption and CO_2 emission. As mentioned earlier, the simulation was done over different test cases in which the number of vehicles was increased consecutively. The purpose of doing that was to see the behavior of the tested algorithms with different traffic states.

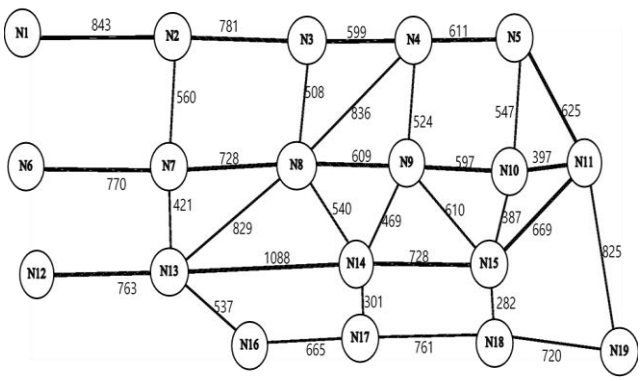

Figure 6: Road Map

4.2.1 Proposed Framework VS Dijkstra's Algorithm

Simulations were carried out in different scenarios and each scenario associated with its number of vehicles. At each scenario we collected data including travel time, CO_2 emission and fuel consumptions. Tables 5, 6, 7 show the simulation results after the different scenarios. Table 5 shows that the proposed framework reduced the travel time and outperformed Dijkstra's algorithm with an improvement up to 19%. Table 6 also presents the Fuel consumption results. The proposed framework showed an improvement up to 24% after different scenarios. Finally, Table 7 also presents the CO_2 emission collected at each simulation scenario. The results show that the proposed framework outperformed Dijkstra's algorithm by reducing the amount of CO_2 emitted by each vehicle during peak time.

Table 5: Travel Time

Total Travel Time (secs)			
Vehicle Number	**Dijkstra's Algorithm**	**Proposed Framework**	**Improvement**
100	449	444	1%
500	385	381	1%
1000	489	457	7%
2000	546	480	14%
3000	594	498	19%
4000	603	524	15%
5000	611	534	14%

Table 6: Fuel Consumption

Total Fuel Consumption (ml)			
Vehicle Number	**Dijkstra's Algorithm**	**Proposed Framework**	**Improvement**
100	463.05	457.32	1%
500	403.10	397.80	1%
1000	509.20	474.78	7%
2000	616.32	531.26	16%
3000	608.67	492.87	23%
4000	658.96	532.96	24%
5000	637.15	532.29	20%

Table 7: CO_2 Emission

Total CO_2 Emission(g/s)			
Vehicle Number	**Dijkstra's Algorithm**	**Proposed Framework**	**Improvement**
100	1077.47	1074.48	0%
500	937.01	914.40	2%
1000	1184.90	1106.72	7%
2000	1431.78	1190.40	20%
3000	1416.51	1240.02	14%
4000	1534.03	1283.80	19%
5000	1481.00	1270.92	17%

5 CONCLUSIONS

This paper proposed an efficient smart transportation platform for solving traffic congestion caused by private transportation. The proposed platform can be served as a blueprint for creating and developing smart transportation applications in smart cities. The proposed platform consists of two sub frameworks, smart parking framework, and optimal routing framework which was used Clean Architecture methodology. In order to evaluate the quality of the smart parking framework, QMOOD methodology is used to calculate the quality metrics in terms of understanding, flexibility, functionality, scalability, efficiency, and reusability. The design metric of the proposed smart parking framework was also compared with the design metric of the SPS proposed in [1], and the results showed a significant improvement of 93.91%

flexibility, 100% scalability, and 90.24% effectiveness. In addition, functionality, and reusability, also improved to 21.6% and 3.2%, respectively. In order to evaluate the performance of the proposed optimal routing framework, SUMO Simulator and TraCI4Matlab were used. The simulation results showed that the optimal routing framework outperformed Dijkstra algorithm. The main purpose of the optimal routing framework was to reduce the traffic congestion and improve transportation system during peak time. The evaluation results also show that the proposed optimal routing platform can reduce the vehicle travel time, fuel consumption and CO_2 emission significantly.

ACKNOWLEDGEMENTS

This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education(NRF-2016R1D1A1B01016073)

REFERENCES

[1] S. C. Yadavalli, *Smart Parking System*, Master's degree thesis, Kansas State University, Manhattan, Kansas, USA, 2016.

[2] M. Pečar and G. Papa, Transportation Problems and Their Potential Solutions in Smart Cities. *2017 International Conference on Smart Systems and Technologies*, IEEE, 2017.

[3] I. Aydin, M. Karakose and E. Karakose, A Navigation and Reservation Based Smart Parking Platform Using Genetic Optimization for Smart Cities, *The 5th International Istanbul Smart Grid and Cities Congress and Fair*, IEEE, pp. 120-124, 2017.

[4] M. Lubis, R. Fauzi, A.R. Lubis and R. Fauzi, Analysis of Project Integration on Smart Parking System in Telkom University, *The 6th International Conference on Cyber and IT Service Management (CITSM 2018)*, Medan, August 6-7, 2018.

[5] A. Khanna and R. Anand, IoT based Smart Parking System. *2016 International Conference on Internet of Things and Applications (IOTA) Maharashtra Institute of Technology*, Pune, India 22 Jan – 24 Jan, pp. 266-270,2016.

[6] R. C. Martin, *Clean architecture: a craftsman's guide to software structure and design.* Prentice Hall Press, pp. 195-199, 2017.

[7] S. N Shukla and T. A Champaneria, Survey of Various Data Collection Ways for Smart transportation Domain of Smart City. *International Conference on IoT in Social, Mobile, Analytics and Cloud*, IEEE, pp. 681-685, 2017.

[8] M. Balta and I. Özçelic, Traffic Signaling Optimization for Intelligent and Green Transportation in Smart Cities, *The 3rd International Conference on Computer Science and Engineering*, IEEE, pp. 31-35, 2018.

[9] S. Shukla, K. Balachandran and V.S. Sumitha, A Framework for Smart Transportation using Big Data. *2016 International Conference on ICT in Business Industry & Government*, IEEE, pp. 1-3, 2016.

[10] K. Chopra, P. Moun, Kapil, Smart Transportation Recommender System, *The 2nd International Conference on Telecommunication and Networks*, IEEE, pp.1-5, 2017.

[11] J. Bansiya and C. G. Davis, A Hierarchical Model for Object-oriented Design Quality Assessment. *IEEE Transactions on software engineering*, IEEE, pp. 4-17, Jan 2002.

[12] Geotab,What is Smart Mobility. Available: https://www.geotab.com/blog/what-is-smart-mobility/. [Online; accessed: December 20, 2018].

[13] A. F. Acosta, J. E. Espinosa, and J. Espinosa, TraCI4matlab: Enabling the Integration of the SUMO Road Traffic Simulator and Matlab Through a Software Re-engineering Process. In Modeling Mobility with Open Data, *The 2nd SUMO Conference 2014 Berlin*, pp. 155-170, Springer, Germany, 2014.

[14] Simulator for Urban Mobilitiy, Available: https://sumo.dlr.de/index.html [Online; accessed: March 21, 2019].

[15] Dynamic Programming, Available: https://en.wikipedia.org/wiki/Dynamic programming [Online; accessed: April 3, 2019].

[16] P. K. Goyal and G. Joshi, "QMOOD Metric Sets to Assess Quality of Java Program", *2014 International Conference on Issues and Challenges in Intelligent Computing Techniques (ICICT)*, IEEE, pp. 520-533, 2014.

[17] SDMetrics, Available: https://www.sdmetrics.com/index.html [Online; accessed: March 16, 2019].

[18] TraCI4Matlab, Available: https://www.mathworks.com/matlabcentral/fileexchange/44805-traci4matlab [Online; accessed: February 20, 2019].

[19] M. El-Wakil, A. El-Bastawisi, M. Boshra and A. Fahmy, Object-Oriented Design Quality Models, A survey and Comparison. *The 2nd International Conference on Informatics and Systems (INFOS04)*, January 2004.

[20] V. Prabhakar, S. Gupta and R. Mehrotra, Smart Transportation for Smart Cities, Available: https://economictimes.indiatimes.com/news/economy/infrastructure/smart-transportation-for-smart-cities/articleshow/48772473.cms?from=mdr [Online; accessed: April 12, 2019].

[21] N.S. Altman, An Introduction to Kernel and Nearest-Neighbor Nonparametric Regression. *1992 The American Statistical Association*, pp. 175-185, 1992.